

1. Johdanto

Monitorointi on valvontaa, joka tarkoittaa verkkojen, palvelimien ja sovellusten tilan jatkuvaa seurantaa, mittaamista ja analysointia. Sillä kerätään havainnointidataa järjestelmän suorituskyvystä.

Manuaalinen tarkistaminen ei ole mahdollista, kun järjestelmät monimutkaistuvat, joten monitorointi auttaa tässä. Monitoroinnin avulla muun muassa häiriöt pystytään havaitsemaan ja korjaamaan nopeasti ja näin ongelmat pystytään tunnistamaan ennen kuin ne vaikuttavat loppukäyttäjiiin. Seurannalla havaitaan esimerkiksi levytilan täyttyminen tai muistin loppuminen ja näin resursseja voidaan lisätä ajoissa. Monitoroinnilla voidaan myös varmistaa, että palvelut täyttävät esimerkiksi saavutettavuuden vaatimukset. Myös vianjäljityksen kannalta monitorointi on tärkeää. Historiadatan avulla voidaan selvittää laitteen tai sovelluksen katkoksen juurisyy.

Prometheus on avoimen lähdekoodin valvonta- ja hälytysjärjestelmä, joka on suunniteltu erityisesti nykyaikaisiin, dynaamisiin ja kontitettuihin ympäristöihin. Sen tärkeimmät ominaisuudet ovat aikasarjatietokanta (TSDP), joka tallentaa datan numeerisina aikasarjoina, PromQL-kyselykieli, joka on tehokas kerätyn datan suodattamiseen ja analysointiin sekä Pull-malli, jossa Prometheus hakee itse metriikat kohteista sen sijaan, että kohteet lähettäisivät sille dataa.

Prometheuksen toiminta perustuu nelivaiheiseen prosessiin. Jos sovellus ei osaa natiivisti tarjota Prometheus-metriikoita, väliin asennetaan Exporter. Prosessin ensimmäinen vaihe on datan keruu (pull), jossa palvelin lukee konfiguraatiostaan valvottavat kohteet (targets) ja hakee http get -pyynnöllä niiden metriikat säännöllisin väliajoin. Toisena tallennus (TSDP), jossa haettu data tallennetaan Prometheuksen omaan aikasarjatietokantaan. Jokaiselle mittaustulokselle liitetään aikaleima ja tunnisteet (labels). Tämän jälkeen kyselyt (PromQL) ja visualisointi (hälytykset). Visualisoinnissa käytetään Grafanaa, joka tekee PromQL-kyselyitä Prometheuksen tietokantaan.

2. Node Exporterin käyttöönotto

(Selitys mitä tehtiin + kuvakaappaukset gittiin) (kuva: node_exporter_metrics)

Asenettiin Node Exportterin versio 1.12.1 Linuxamd64:lle. Asensimme myös wget ja tar. Wgetin avulla pystymme latamaan HTTPS-alkuisia osoitteita suoraan komentoriviltä. Tässä tapauksessa se oli <https://github.com> ... eli saimme ladattua suoraan githubista uusimman version Node Exportterista. Sitten purimme tiedostot käyttämällä tar xvf node_exporter-*.linux-amd64.tar.gz. Sitten siirymme Node Exportteriin hakemistoon commandillä cd node_exporter-1.12.1.linux-amd64 - komennolla. Hakemistossa käynnistimme Node Exportterin ./node_exporter - komennolla. Sen jälkeen avasimme uuden terminaalin ja käynnistimme ympäristön, jonne pistimme käskyn curl <http://localhost:9100/metrics>. Tuloksena tuli valtava määrä informaatiota eri parametreista.

3. Prometheus

Avasimme Prometheusin ja menimme target health -osioon, josta rajasimme haun web1 -nimisiin kohteisiin. Web1 näkyi kohteena ja tila oli up. Tämän lisäksi näkyi webapps. Kuvakaappaukset gittiin lisättiin nimellä prometheus_targets.

4. Grafanan tietolähde

Menimme Grafanan add new connection ja lisäsimme sinne Prometheusin. Sen jälkeen lisäsimme <http://prometheus:9090> url-osoitteen ja suoritimme onnistuneesti yhteydenoton. Kuvakaappaus lisättiin gittiin nimellä grafanan_tietolahde.

5. Dashbord

Selitys visualisoiduista mittareista, perustele mittareiden hyödyllisyys? + kuvakaappaukset (kuva: grafana_dashboard)

CPU Usage %, seuraa koneen CPU:n käyttöä prosentuaallisesti. Esimerkiksi, jos kone pyörittää isoa peliä / tiedostoa tai tekee jotain suurta mikä kuluttaa paljon CPU:ta, CPU Usage % nousee. Esimerkiksi, jos CPU % on 5%, kone ei kuormita itseään hirveästi, mutta jos se nousee vaikka yhtäkkisesti 90%, jotain on pielessä ja voi alkaa syntyä eri ongelmia, kuten kaatumisia ja koneen hidastumista.

Memory Usage % kertoo koneen muistin, eli RAM:in tilasta ja kuinka paljon sitä on käytössä ja käytetty. Jos RAM täyttyy liikaa, syntyy samanlaisia ongelmia kuin CPU:n käytön kanssa, eli kaatumisia ja koneen hidastumista.

Disk Usage % kertoo levyn käyttöasteen prosentteina. Siitä selviää kuinka suuri osa järjestelmän kiintolevystä on varattu suhteessa sen kokonaismäärään. Jos lukema on esimerkiksi 75 %, se tarkoittaa että 75 % levytilasta sisältää dataa. Jos levy on 100 % täynnä se tarkoittaa, ettei järjestelmä pysty enää kirjoittamaan uutta dataa, siitä saattaa seurata esimerkiksi sovellusten kaatumista ja/tai sitä, ettei järjestelmään pääse enää kirjautumaan sisään.

Network Receive kertoo, paljonko verkolle saapuu dataa. Esimerkiksi erilaiset lataukset (opettajan ohjeiden lataaminen Moodlesta) ovat verkon saapuvaa liikennettä.

Network Transmit kertoo taas, kuinka paljon laite lähettää verkolle dataa. Esimerkiksi tiedostojen lähetys (opettajan tehtävän palautus Moodleen) on verkolle lähtevää liikennettä.

6. Kuormitustesti

(Miten CPU-käyrä muuttui? Miten levytilan käyttö muuttui? Näkyikö verkkoliikenteessä muutoksia? + kuvakaappaukset gittiin) (kuva: load_test)

CPU-käyrä nousi kaikilla tasaisesti kolmeen kymmeneen (client, web1, db1).

Memory Usage-käyrä nousi ensin nopeasti 44, sitten pysähtyi hetkeksi, nousi uudestaan nopeasti 45,5, jonka jälkeen se laski hetkeksi, kunnes nousi tasaisesti 46:n. Kaikkien käyrä pysyi hetken aikaa rajapinnassa, jonka jälkeen se laski äkisti 43:n. Ja se tapahtui myös kaikilla (client, web1, db1)

Disk-käyrä nousi nopeasti kaikilla 6.22 ja pysyi siellä. (Web1, client, db1)

Verkkoliikenteessä molemmissa (receive ja transmit) nousi yksi käyrä selkeästi, eli web1: n käyrä.

Kaikki käyrät siis nousivat rajakattoon, mutta verkkoliikenteessä vain web1: llä oli muutoksia.

7. SNMP vs Prometheus

(Vertaa viikon 2 SNMP-ratkaisua viikon 3 Prometheus-ratkaisuun)

Täydennä taulukko

Ominaisuus	SNMP	Prometheus
Tiedonkeruu	Laaja valikoima, kunhan tietää mitä etsii.	Erittäin hyvä keräämään aktiivista tietoa, jonka pystyy myös selkeästi havaitsemaan graafisesti.
Käyttöönotto	Oli vaikeaa ja työlästä.	Oli helppoa ja hauskaa.
Mittarien määrä	Paljon mittareita eri kohteista.	Mittareita on paljon, mutta tiettyjen mittareiden saamiseen tarvitaan tarkkoja komentoja.
Visualisointi	Vaikea havainnoida, ellei ole pyytänyt juuri tiettyä esim. SysName.0 kommentia.	Paljon selkeämpi ja helpompi havainnoida visuaalisesti, mitä tapahtuu.
Hälytysmahdollisuudet	Komentojen asettaminen on hankalaa ja työlästä, eivätkä ne tule automaattisesti.	Helppo havainnoida visuaalisesti heti, käyrät näyttävät poikkeavuudet. Myös hälytysten automatisointi vaikuttaa helpommalta, kuin SNMP:ssä.
Soveltuvuus pilviympäristöihin	Jos sen kanssa jaksaa vääntää, muuten erittäin työläs ja kankea.	Toimii varmasti erinomaisesti.

Pohdinta ja yhteenveto

Mitä hyötyä Prometheusesta on verrattuna SNMP:hen?

Prometheus käyttää labeleita datan luokitteluun, kun taas SNMP perustuu laitekohtaisiin OID-tunnisteisiin. Prometheus kykenee suodattamaan ja ryhmittämään metriikkaa paljon joustavammin, koska se on nimenomaan suunniteltu erityisesti nykyaikaisempien pilvipalveluiden, kontitettujen ympäristöjen ja sovellusten valvontaan.

Millaisia mittareita ylläpitäjän kannattaa seurata jatkuvasti?

Ylläpitäjän kannattaa seurata jatkuvasti sovellusten ja palveluiden mittareista latausnopeutta, liikennettä, virheiden määrää ja jonoja. Infrastruktuurin ja palvelinten mittareista CPU:n käyttöastetta, muistinkäyttöä, levytilaa ja verkkoliikennettä. Jotta Grafanan ja Prometheuksen toimintaan voi luottaa, pitää valvoa myös itse valvontajärjestelmän osalta kohteiden tilaa, sekä scrape duration ja errors mittareita.

Mitä tietoa dashboardisi tarjoaa ylläpitäjälle?

Dashboard tarjoaa ylläpitäjille reaaliaikaista tietoa resursseista ja kuormituksesta, muun muassa resurssien käyttöasteesta ja suorituskyvystä (CPU usage, memory usage, disk usage) sekä verkkoliikenteestä ja verkon kuormituksesta (network receive, network transmit).

Mitä uusia mittareita lisäisit dashboardiin?

Mittarit olivat Memory Usage%, CPU Usage%, Disk Usage%, Network Recieve ja Network Transmit

Miten monitorointitiedosta voisi olla hyötyä vianetsinnässä?

Monitoroinnin avulla ongelman pystyy paikantamaan helpommin, esim jos sovellus vastaa hitaasti, monitorointi näyttää heti johtuuko se esimerkiksi CPU:n ylikuormituksesta. Prometheuksen metriikan avulla pystytään nähdä, koskeeko vika kaikkia palvelimia vai vain yhtä tiettyä nodea. Monitoroinnista voidaan myös tarkastella mitä tapahtui juuri ennen katkoa, ja sitä kautta selvittää miten aiempi toiminta on voinut vaikuttaa ongelman syntymiseen.

Mitä opittiin?

Opimme, että SNMP verrattuna Grafanan ja Prometheuksen käyttö oli paljon helpompaa ja selkeämpi ymmärtää. Käyttö sujui paremmin, koska mittarit olivat visuaalisessa muodossa ja niitä pystyi hahmottamaan paljon paremmin.